

DIY-Spektrometer mit ESP32-CAM (Optischer Ansatz)

Übersicht

Diese Anleitung beschreibt den Bau eines kostengünstigen DIY-Spektrometers unter Verwendung eines ESP32-CAM-Moduls als Detektor. Das Spektrometer nutzt ein Beugungsgitter (z. B. eine CD/DVD-Oberfläche oder ein lineares Gitter) zur Zerlegung des Lichts in sein Spektrum. Die ESP32-CAM erfasst das Spektrum, und eine Software analysiert die Pixelintensität entlang der Spektrallinie. Die Daten werden über Home Assistant direkt in die NIR-Plattform eingespeist.

Komponentenliste

Hardware

Komponente	Beschreibung	Empfohlene Quelle
ESP32-CAM-Modul	Kamera mit OV2640- oder OV3660-Sensor (3MP, 160 Grad Weitwinkel, Infrarot-Unterstützung)	DFRobot (DFR1154), AliExpress, Amazon
Beugungsgitter	CD/DVD-Oberfläche (kostengünstig) oder lineares Gitter (höhere Präzision)	Alte CDs/DVDs, Optik-Händler
Schlitzblende	Präziser Schlitz (z. B. aus Rasierklingen oder 3D-gedruckt)	Selbstgebaut oder 3D-Druck
Gehäuse	Lichtdichtes Gehäuse zur Fixierung der Komponenten	3D-gedruckt (STL-Dateien siehe Anhang)
Stromversorgung	5V USB-C oder 3.7–15V DC (je nach ESP32-CAM-Modell)	USB-Netzteil, Powerbank
Kabel & Stecker	Jumper-Kabel, MicroSD-Karte (optional für Datenspeicherung)	Elektronik-Händler

Software

Komponente	Beschreibung	Link
Arduino IDE	Entwicklungsumgebung für ESP32	Download
ESP32-Board-Support	Board-Definitionen für ESP32 in Arduino IDE	Anleitung
OpenCV (optional)	Bildverarbeitung zur Spektrumanalyse	OpenCV
Home Assistant	Automatisierungsplattform zur Datenintegration	Home Assistant

Aufbau des Spektrometers

Schritt 1: Vorbereitung des Beugungsgitters

1. CD/DVD als Gitter nutzen:
 - Entferne die reflektierende Schicht einer alten CD/DVD mit einem Cuttermesser oder Schleifpapier, um eine klare Gitterstruktur freizulegen.
 - Alternativ: Lineares Beugungsgitter (z. B. 1000 Linien/mm) für höhere Präzision verwenden.
 - Hinweis: CD/DVDs haben ca. 600–900 Linien/mm und eignen sich für den sichtbaren Bereich (400–700 nm).

2. Positionierung des Gitters:

- Das Gitter sollte im 45-Grad-Winkel zur einfallenden Lichtquelle platziert werden, um eine optimale Spektralaufspaltung zu erreichen.
- Fixiere das Gitter in einem 3D-gedruckten Halter oder klebe es in einem stabilen Winkel.

Schritt 2: Schlitzblende herstellen

1. Material:

- Verwende zwei Rasierklingen oder dünne Metallplättchen, die mit einem Abstand von 0,1–0,5 mm zueinander fixiert werden.
- Alternativ: 3D-Druck einer Schlitzblende mit präziser Öffnung.

2. Montage:

- Die Schlitzblende sollte direkt vor dem Kameramodul oder in einem Abstand von 5–10 cm platziert werden, um eine scharfe Spektrallinie zu erzeugen.
- Empfehlung: Teste verschiedene Schlitzbreiten, um die beste Auflösung zu erzielen.

Schritt 3: Gehäuse konstruieren

1. 3D-Druck:

- Nutze die beigefügten STL-Dateien für ein lichtdichtes Gehäuse.
- Anforderungen:
 - Lichtdichte Abdeckung (außer Schlitz und Kameralinse).
 - Fixierung für ESP32-CAM, Gitter und Schlitzblende.
 - Optional: Halterung für eine Lichtquelle (z. B. LED für Kalibrierung).

2. Manuelle Alternative:

- Verwende eine lichtdichte Box (z. B. aus Pappe oder Holz) und klebe die Komponenten in der richtigen Ausrichtung ein.

3. Anordnung der Komponenten:

[Lichtquelle] -> [Schlitzblende] -> [Beugungsgitter] -> [ESP32-CAM]

- Abstände:
 - Schlitzblende zu Gitter: 5–10 cm
 - Gitter zu Kamera: 10–20 cm (je nach Brennweite der Kamera)

Schritt 4: ESP32-CAM anschließen

1. Stromversorgung:

- Verbinde das ESP32-CAM-Modul über USB-C oder den VIN-Pin (3.7–15V) mit einer Stromquelle.
- Wichtig: Die Kamera benötigt ausreichend Strom (mind. 500 mA).

2. Programmierung:

- Verbinde das ESP32-CAM über einen USB-Serial-Adapter (z. B. CP2102) mit dem PC.
-

Software-Implementierung

Schritt 1: Arduino IDE einrichten

1. Board-Support hinzufügen:

- Öffne die Arduino IDE und füge die ESP32-Board-URL hinzu: https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
- Installiere das Paket über Tools > Board > Boards Manager (Suchbegriff: esp32).

2. ESP32-CAM auswählen:

- Wähle das Board ESP32S3 Dev Module oder AI Thinker ESP32-CAM (je nach Modell).
- Partition Scheme: Huge APP (3MB No OTA/1MB SPIFFS) oder Default.

3. Serielle Schnittstelle konfigurieren:

- Wähle den korrekten COM-Port (z. B. /dev/ttyUSB0 oder COM3).
- Upload Method: UART (für DFR1154: USB OTG).

Schritt 2: Kamera initialisieren

```
#include "esp_camera.h"
#include "Arduino.h"
// Kamera-Pins für ESP32-S3 AI CAM (DFR1154)
#define XCLK_GPIO_NUM 5
#define SIOD_GPIO_NUM 8
#define SIOC_GPIO_NUM 9
#define Y9_GPIO_NUM 4
#define Y8_GPIO_NUM 6
#define Y7_GPIO_NUM 7
#define Y6_GPIO_NUM 14
#define Y5_GPIO_NUM 17
#define Y4_GPIO_NUM 21
#define Y3_GPIO_NUM 18
#define Y2_GPIO_NUM 16
#define VSYNC_GPIO_NUM 1
#define HREF_GPIO_NUM 2
#define PCLK_GPIO_NUM 15
void setup() {
  Serial.begin(115200);
  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer = LEDC_TIMER_0;
  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
```

```

config.pin_d6 = Y8_GPIO_NUM;
config.pin_d7 = Y9_GPIO_NUM;
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sscb_sda = SIOD_GPIO_NUM;
config.pin_sscb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = -1;
config.pin_reset = -1;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_GRAYSCALE;
config.frame_size = FRAMESIZE_UXGA;
config.jpeg_quality = 10;
config.fb_count = 2;
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Kamera-Initialisierung fehlgeschlagen: 0x%x", err);
    return;
}
Serial.println("Kamera bereit!");
}
void loop() {
    camera_fb_t *fb = esp_camera_fb_get();
    if (!fb) {
        Serial.println("Fehler beim Erfassen des Bildes");
        return;
    }
    esp_camera_fb_return(fb);
    delay(1000);
}

```

Schritt 3: Spektrumanalyse

```

#include <vector>
#include <algorithm>
std::vector<uint8_t> analyzeSpectrum(camera_fb_t *fb) {
    std::vector<uint8_t> spectrum;
    int startX = 100;
    int endX = fb->width - 100;
    int y = fb->height / 2;
    for (int x = startX; x < endX; x++) {
        uint8_t pixel = fb->buf[y * fb->width + x];
        spectrum.push_back(pixel);
    }
    return spectrum;
}
std::vector<float> calibrateWavelengths(int spectrumLength) {
    std::vector<float> wavelengths;
    float startWavelength = 400.0;

```

```

float endWavelength = 700.0;
for (int i = 0; i < spectrumLength; i++) {
    float wavelength = startWavelength + (endWavelength -
startWavelength) * (i / (float)spectrumLength);
    wavelengths.push_back(wavelength);
}
return wavelengths;
}

```

Schritt 4: Datenverarbeitung und Kalibrierung

1. Kalibrierung mit bekannter Lichtquelle (z. B. Neonlampe mit Linien bei 656 nm, 585 nm, etc.).
2. Datenfilterung:
 - Moving Average:

```

std::vector<uint8_t> smoothSpectrum(std::vector<uint8_t> spectrum, int
windowSize = 5) {
    std::vector<uint8_t> smoothed;
    for (int i = 0; i < spectrum.size(); i++) {
        int sum = 0;
        int count = 0;
        for (int j = std::max(0, i - windowSize/2); j <=
std::min((int)spectrum.size()-1, i + windowSize/2); j++) {
            sum += spectrum[j];
            count++;
        }
        smoothed.push_back(sum / count);
    }
    return smoothed;
}

```

Schritt 5: Datenübertragung an Home Assistant

```

#include <WiFi.h>
#include <HTTPClient.h>
const char* ssid = "DEIN_WIFI_SSID";
const char* password = "DEIN_WIFI_PASSWORD";
const char* homeAssistantUrl = "http://DEINE_HOME_ASSISTANT_IP:8123/api/states/
sensor.spektrometer";
const char* homeAssistantToken = "DEIN_LONG_LIVED_TOKEN";
void sendToHomeAssistant(std::vector<uint8_t> spectrum, std::vector<float>
wavelengths) {
    HTTPClient http;
    String jsonData = "{";
    jsonData += "\"wavelengths\":[";
    for (int i = 0; i < wavelengths.size(); i++) {
        jsonData += String(wavelengths[i]);
    }
}

```

```

if (i < wavelengths.size() - 1) jsonData += ",";
}
jsonData += "],";
jsonData += "\"intensities\":[\";
for (int i = 0; i < spectrum.size(); i++) {
jsonData += String(spectrum[i]);
if (i < spectrum.size() - 1) jsonData += ",";
}
jsonData += "]]";
http.begin(homeAssistantUrl);
http.addHeader("Authorization", "Bearer " + String(homeAssistantToken));
http.addHeader("Content-Type", "application/json");
int httpCode = http.POST(jsonData);
if (httpCode > 0) {
Serial.printf("Daten gesendet (HTTP %d)\n", httpCode);
} else {
Serial.printf("Fehler beim Senden: %s\n", http.errorToString(httpCode).c_str());
}
http.end();
}

```

Schritt 6: Home Assistant Konfiguration

```

# configuration.yaml
rest_command:
  spektrometer_update:
    url: "http://DEINE_ESP32_IP:80/spectrum"
    method: POST
    payload: "{{ payload }}"
sensor:
  - platform: rest
    name: "Spektrometer Daten"
    resource: "http://DEINE_ESP32_IP:80/spectrum"
    value_template: "OK"
    scan_interval: 60

```

NIR-Plattform-Integration

```

# NIR-Plattform-Addon Konfiguration
spektrometer:
  sensor_id: sensor.spektrometer_daten
  wavelength_range: [400, 700]
  update_interval: 60

```

Vollständiger Beispielcode

```
#include "esp_camera.h"
#include <WiFi.h>
#include <HTTPClient.h>
#include <vector>
// Kamera-Pins (ESP32-S3 AI CAM)
#define XCLK_GPIO_NUM 5
#define SIOD_GPIO_NUM 8
#define SIOC_GPIO_NUM 9
#define Y9_GPIO_NUM 4
#define Y8_GPIO_NUM 6
#define Y7_GPIO_NUM 7
#define Y6_GPIO_NUM 14
#define Y5_GPIO_NUM 17
#define Y4_GPIO_NUM 21
#define Y3_GPIO_NUM 18
#define Y2_GPIO_NUM 16
#define VSYNC_GPIO_NUM 1
#define HREF_GPIO_NUM 2
#define PCLK_GPIO_NUM 15
const char* ssid = "DEIN_WIFI_SSID";
const char* password = "DEIN_WIFI_PASSWORD";
const char* homeAssistantUrl = "http://DEINE_HOME_ASSISTANT_IP:8123/api/states/
sensor.spektrometer";
const char* homeAssistantToken = "DEIN_LONG_LIVED_TOKEN";
std::vector<uint8_t> analyzeSpectrum(camera_fb_t *fb) {
    std::vector<uint8_t> spectrum;
    int y = fb->height / 2;
    int startX = 100;
    int endX = fb->width - 100;
    for (int x = startX; x < endX; x++) {
        spectrum.push_back(fb->buf[y * fb->width + x]);
    }
    return spectrum;
}
std::vector<float> calibrateWavelengths(int spectrumLength) {
    std::vector<float> wavelengths;
    float startWavelength = 400.0;
    float endWavelength = 700.0;
    for (int i = 0; i < spectrumLength; i++) {
        wavelengths.push_back(startWavelength + (endWavelength -
startWavelength) * (i / (float)spectrumLength));
    }
    return wavelengths;
}
std::vector<uint8_t> smoothSpectrum(std::vector<uint8_t> spectrum, int
windowSize = 5) {
    std::vector<uint8_t> smoothed;
    for (int i = 0; i < spectrum.size(); i++) {
        int sum = 0;
        int count = 0;
```

```

    for (int j = std::max(0, i - windowSize/2); j <= std::min((int)spectrum.size()-1,
i + windowSize/2); j++) {
        sum += spectrum[j];
        count++;
    }
    smoothed.push_back(sum / count);
}
return smoothed;
}

void sendToHomeAssistant(std::vector<uint8_t> spectrum, std::vector<float>
wavelengths) {
    HTTPClient http;
    String jsonData = "{\"wavelengths\":[";
    for (int i = 0; i < wavelengths.size(); i++) {
        jsonData += String(wavelengths[i]);
        if (i < wavelengths.size() - 1) jsonData += ",";
    }
    jsonData += "],\"intensities\":[";
    for (int i = 0; i < spectrum.size(); i++) {
        jsonData += String(spectrum[i]);
        if (i < spectrum.size() - 1) jsonData += ",";
    }
    jsonData += "]}";
    http.begin(homeAssistantUrl);
    http.addHeader("Authorization", "Bearer " + String(homeAssistantToken));
    http.addHeader("Content-Type", "application/json");
    int httpCode = http.POST(jsonData);
    if (httpCode > 0) {
        Serial.printf("Daten gesendet (HTTP %d)\n", httpCode);
    } else {
        Serial.printf("Fehler beim Senden: %s\n", http.errorToString(httpCode).c_str());
    }
    http.end();
}

void setup() {
    Serial.begin(115200);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("WiFi verbunden!");
    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM;
    config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM;
    config.pin_d7 = Y9_GPIO_NUM;
    config.pin_xclk = XCLK_GPIO_NUM;

```



```

config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sscb_sda = SIOD_GPIO_NUM;
config.pin_sscb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = -1;
config.pin_reset = -1;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_GRAYSCALE;
config.frame_size = FRAMESIZE_UXGA;
config.jpeg_quality = 10;
config.fb_count = 2;
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Kamera-Initialisierung fehlgeschlagen: 0x%x", err);
    return;
}
Serial.println("Kamera bereit!");
}

void loop() {
    camera_fb_t *fb = esp_camera_fb_get();
    if (!fb) {
        Serial.println("Fehler beim Erfassen des Bildes");
        return;
    }
    std::vector<uint8_t> spectrum = analyzeSpectrum(fb);
    std::vector<float> wavelengths = calibrateWavelengths(spectrum.size());
    std::vector<uint8_t> smoothedSpectrum = smoothSpectrum(spectrum);
    sendToHomeAssistant(smoothedSpectrum, wavelengths);
    esp_camera_fb_return(fb);
    delay(5000);
}

```

Optimierung und Fehlerbehebung

Häufige Probleme

Problem	Ursache	Lösung
Kein Spektrum sichtbar	Falsche Ausrichtung des Gitters	Gitter im 45-Grad-Winkel neu ausrichten
Unscharfe Spektrallinie	Schlitzblende zu breit	Schlitzbreite auf 0,1–0,3 mm reduzieren
Geringe Intensität	Unzureichende Beleuchtung	Hellere Lichtquelle verwenden
Kamera erfasst kein Bild	Falsche Pinbelegung	Pinbelegung für das spezifische ESP32-CAM-Modell prüfen
WiFi-Verbindung bricht ab	Strommangel	Externes Netzteil mit ausreichend Strom verwenden

Tipps zur Verbesserung

- Kalibrierung: Verwende eine Neonlampe mit bekannten Spektrallinien.
- Rauschen reduzieren: Mehrfachmessungen und Filter (Gauss, Moving Average) anwenden.

- Lichtquelle: Für NIR-Bereich (700–1100 nm) eine Infrarot-LED (850 nm oder 940 nm) verwenden.
 - Gitterauswahl: Lineares Gitter (1000 Linien/mm) für höhere Auflösung.
-

3D-Druckvorlagen (STL-Dateien)

- Gehäuse: spektrometer_gehaeuse.stl (150 × 100 × 80 mm)
 - Gitterhalterung: gitter_halterung.stl (45-Grad-Neigung)
 - Schlitzblenden-Halter: schlitzeblende_halter.stl (einstellbare Schlitzbreite)
-

Sicherheitshinweise

- Verwende nur isolierte Netzteile mit der richtigen Spannung.
 - Vermeide direkte Sonneneinstrahlung auf die Kamera.
 - Achte auf lichtdichtes Gehäuse, um Streulicht zu vermeiden.
-

Ressourcen

- [ESP32-CAM Dokumentation](#)
 - [Home Assistant REST API](#)
 - [NIR-Plattform \(GitHub\)](#)
 - [DIY Spectrometry Group \(Public Lab\)](#)
-

Anhang: Pinbelegungen

ESP32-CAM (AI Thinker)

Funktion	GPIO-Pin
XCLK	0
Y9	2
Y8	4
Y7	12
Y6	13
Y5	14
Y4	15
Y3	16
Y2	17
VSYNC	5
HREF	27
PCLK	25

ESP32-S3 CAM (DFRobot DFR1154)

Funktion	GPIO-Pin
XCLK	5
Y9	4
Y8	6
Y7	7
Y6	14
Y5	17
Y4	21
Y3	18
Y2	16
VSYNC	1
HREF	2
PCLK	15